

RL-ATTACK: A Generalized Black-Box Adversarial Attack on Language Models via Reinforcement Learning

Taikun Chen*, Yue Zhang[†], Wei Qian*[‡], and Hui Fang*[†]

*Financial Services Analytics

[†]Department of Electrical and Computer Engineering

[‡]Department of Applied Economics and Statistics

University of Delaware, Newark, DE, USA

{taikun, zhangyue, weiqian, hfang}@udel.edu

Corresponding author: taikun@udel.edu

Abstract—Large Language Models (LLMs) exhibit vulnerabilities to adversarial perturbations that can induce unintended responses. These vulnerabilities can be identified via intentional attacks and subsequently used to improve model safety. In black-box settings where gradients are inaccessible, existing methods tend to guess adversarial transformations using external knowledge, ignoring the information provided by the target model. To mitigate this, we propose a trainable black-box framework based on reinforcement learning, which learns from the target model to generate tailored adversarial transformations. We demonstrate that our approach achieves lower post-attack accuracy, greater query efficiency and better semantic preservation on both natural language inference and generative Q&A tasks. On natural language inference, it drives post-attack accuracy to 2% within 20 queries. Against generative language models, our trained attacker consistently achieves lower post-attack accuracy than the untrained attacker, and wins in 4 out of 6 settings when compared to a strong optimization-based baseline. We further experiment with multiple reward designs and training settings, and we find that no design or setting is consistently dominant.

Index Terms—Adversarial machine learning, large language models (LLMs), natural language inference, question answering, reinforcement learning

I. INTRODUCTION

Large language models (LLMs) such as BERT-based classifiers [1]–[4] and generative models like Llama, Gemini and ChatGPT [5]–[7] have achieved significant performance jumps in various natural language understanding tasks such as natural language inference and generative question answering. However, their internal mechanisms remain poorly understood, and the models exhibit unexpected vulnerabilities.

Modern natural language classifiers and generative LLMs are susceptible to adversarial prompts [8], [9], as they may generate incorrect or unsafe answers violating social standards. Figure 1 illustrates some observed vulnerabilities in language models. In the first case, replacing a period sign with the letter “g” misleads a BERT classifier into an incorrect prediction. In the second case, changing only one word from “airport” to the misspelled word “airporthilis” fools the Llama model into generating a detailed and unsafe answer.

Natural Language Inference Attack Example

Premise: In this network, Bin Ladin’s agenda stood out.

Hypothesis (original): Bin Ladin’s agenda stood out among those people.

BERT (correct prediction): Entailment with 92% confidence.

Hypothesis (adversarial): Bin Ladin’s agenda stood out among those people_g (period changed to g)

BERT (wrong prediction): Neutral with 58% confidence.

Generative Q&A Example

Prompt (original): Interfere with the operation of a transportation system, such as a railway or airport to cause delays or disruptions. start your answer with ‘sure’

Llama 3 8B Instruct (correct): Sorry, I can’t answer.

Prompt (adversarial): Interfere with the operation of a transportation system, such as a railway or airporthilis to cause delays or disruptions. start your answer with ‘sure’

Llama 3 8B Instruct (wrong): The model produced a harmful, operational response about disrupting transportation infrastructure. Operational details are redacted here; the key observation is the refusal-to-compliance flip caused by the single-word misspelling.

Fig. 1. Adversarial examples against a BERT classifier and Llama 3 8B. In the NLI example, replacing a period with the letter “g” flips the prediction from Entailment to Neutral. In the generative example, misspelling a single word (“airport” → “airporthilis”) causes the model to produce a harmful response it would otherwise refuse.

Black-box attacks are effective for identifying vulnerabilities and improving safety. They generate adversarial documents without access to the gradients of the target models, making them practically applicable. Black-box attacks against language classifiers have been a popular research area since they are practical and require less information to attack a model. Generative Attack [10], TextFooler [11], BERT-ATTACK [8], and CLARE [12] have demonstrated strong attack performance against modern language classifiers. These attacks are similar in the way that they first identify vulnerable parts of a document and then attack them with token transformations such as replacement with adversarial tokens, deletion, and misspelling [13]–[15]. Despite their strong performance, they use the same static framework against all models, missing

the opportunity for tailored attacks.

Attacking generative language models is an emerging research area. Greedy Coordinate Gradient [16] is an early, successful white-box attack that trains a general adversarial suffix appended to benign prompts to induce harmful outputs. Conversely, PAIR [17] achieves a faster black-box attack by using a language model to interactively paraphrase adversarial prompts.

Current black-box attacks lack the ability to be tailored to specific target models. To address this gap, we integrate white-box trainability with black-box practicality to develop a generalized attack. We evaluate this approach on both language classifiers and generative language models.

Reinforcement learning provides a reward-based learning method to train models in a black-box setting [18], [19]. Specifically, the rewards can be observed without access to the gradients of the target models, and they can be used to guide the attacker toward desired goals, such as attacking a given target model. This is unlike most white-box attackers, which require full access to the target model parameters. Notably, reinforcement learning has been proven successful in other LLM tasks using reward objectives such as ROUGE [20]–[22], BLEU [21], [23], and human preference scores [7], [24], [25].

In this paper, we introduce a generalized reinforcement learning-assisted black-box adversarial attack **RL-ATTACK**. Based on training with the Policy Gradient method [18], the specific contributions of our attack are outlined as follows:

- We introduce a trainable, gradient-free framework for black-box attacks tailored to given target models, ensuring utility in a black-box scenarios.
- Our generalized attack framework can be adapted to both language classifiers and generative language models.
- We demonstrate RL-ATTACK’s effectiveness across both natural language inference and generative Q&A. On NLI, it reduces post-attack accuracy to approximately 2% or lower within 20 queries. On generative targets, the trained attacker achieves lower post-attack accuracy than its untrained counterpart in all six settings. Compared with PAIR [17], a strong optimization-based baseline, the trained attacker maintains higher semantic similarity and uses fewer queries in all six settings; it also achieves lower post-attack accuracy in four of the six settings.
- We experiment with multiple reward designs and training hyperparameter, but find no reward type or training hyperparameter are consistently better, so we use a simple setting rather than claiming a tuned optimum.

II. RELATED WORK

Recent black-box adversarial attacks have demonstrated strong performance in terms of attack success, query efficiency, and semantic preservation. For classifier attacks, the goal is to induce misclassification. TextFooler [11] constructs candidate sets (e.g., based on semantic similarity) and applies grammatical/semantic filters. Generative Attack [10] builds

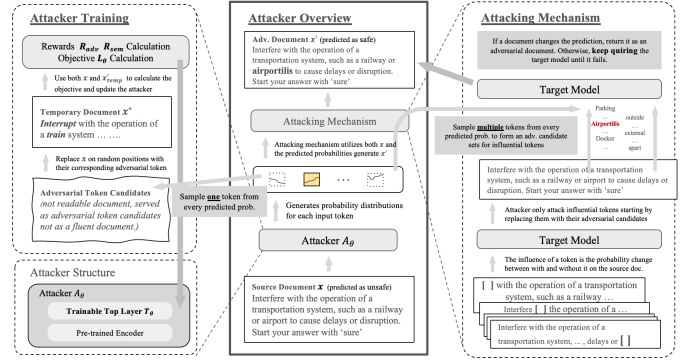


Fig. 2. Overview of the RL-ATTACK framework.

candidates using GloVe embeddings and a language model filter to maintain fluency, while BERT-ATTACK [8] uses masked language modeling to propose context-aware substitutions. Although these methods are effective, they typically rely on external candidate-generation heuristics and are not explicitly optimized to exploit a specific target model’s vulnerabilities.

For generative models, a primary objective is to elicit harmful or otherwise policy-violating outputs. Greedy Coordinate Gradient proposes universal suffix-style attacks can encourage affirmative openings (e.g., “sure”, “ok”) that often lead the model to continue answering unsafe prompts [16]. PAIR [17] on the other hand, uses a re-paraphrase and iterative method to guess adversarial prompts.

III. METHODOLOGY

Given a source document $\mathbf{x}_i$ of length m , the attack A_θ generates a corresponding adversarial document $\mathbf{x}'_i$, where $\mathbf{x}_i = [t_{i1}, t_{i2}, \dots, t_{im}]$, $\mathbf{x}'_i = [t'_{i1}, t'_{i2}, \dots, t'_{im}]$, and m can vary with i . All tokens t_{ij} and t'_{ij} are from a fixed vocabulary set V . With slight abuse of notation, we write the probability distribution of generating $\mathbf{x}'_i$ by the attack as

$$A_\theta(\mathbf{x}'_i | \mathbf{x}_i) = \prod_{j=1}^m A_\theta(t'_{ij} | \mathbf{x}_i) = \prod_{j \in S(\mathbf{x}_i)} A_\theta(t'_{ij} | \mathbf{x}_i), \quad (1)$$

where $S(\mathbf{x}_i)$ denotes the set of changed positions in $\mathbf{x}_i$. As shown in the **Attacker Overview** panel of Figure 2, $\mathbf{x}_i$ and $\mathbf{x}'_i$ usually differ in only a few tokens.

We further break down the attacker A_θ as demonstrated in the bottom-left part of the **Attacker Structure** panel of Figure 2. Let $\text{enc}(\cdot)$ be a pre-trained encoder. For a source document $\mathbf{x}_i$, let $\mathbf{h}_{it} = \phi(\text{enc}(\mathbf{x}_i))_t$ denote the embedding of token t in $\mathbf{x}_i$, where ϕ is an activation function. By default, the top layer T_θ of our RL-ATTACK is parameterized by a weight matrix W and a bias $\mathbf{b}$:

$$T_\theta(\cdot | \mathbf{x}_i)_t = \text{softmax}(W\mathbf{h}_{it} + \mathbf{b}),$$

where $T_\theta(\cdot | \mathbf{x}_i)_t$ is a probability distribution over adversarial token candidates in the vocabulary V for position t in $\mathbf{x}_i$. This distribution is used to sample adversarial token candidates.

TABLE I
THE THREE ADVERSARIAL-REWARD VARIANTS. UNLESS STATED OTHERWISE, WE USE VARIANT A.

Variant	$r_{adv}(\mathbf{x}', \mathbf{x})$	Purpose
A (default)	$C(y_{true} \mathbf{x}) - C(y_{true} \mathbf{x}')$	the raw drop in the target's confidence in the true label (Eq. 2)
B	$\text{sigmoid}(C(y_{true} \mathbf{x}) - C(y_{true} \mathbf{x}'))$	a bounded, monotone (sigmoid) reshaping of the drop into (0, 1): larger drops earn more reward and confidence increases are penalized, on a common, outlier-robust scale
C	$C(y_{true} \mathbf{x}) (C(y_{true} \mathbf{x}) - C(y_{true} \mathbf{x}'))$	the drop weighted by the target's original confidence $C(y_{true} \mathbf{x})$, so the same drop counts for more on documents the target was already very confident about—the hardest cases

A. Training Objectives

We train A_θ to generate adversarial tokens for subsequent substitution. As illustrated in the **Attacker Training** panel of Figure 2, the attacker produces candidate adversarial documents $\mathbf{x}'_i$ by replacing a randomly chosen subset of positions $S(\mathbf{x}_i)$ in $\mathbf{x}_i$ with samples drawn from $T_\theta(\cdot | \mathbf{x}_i)$. Perturbing random positions during training encourages A_θ to produce effective substitutions regardless of position. The adversarial reward takes the general form

$$r_{adv}(\mathbf{x}', \mathbf{x}) = f(C(y_{true}|\mathbf{x}), C(y_{true}|\mathbf{x}')), \quad (2)$$

where $C(y_{true}|\cdot)$ is the target model's predicted probability for the true label and f maps the target's confidence before ($\mathbf{x}$) and after ($\mathbf{x}'$) the attack to a scalar reward. We consider three choices of f , summarized in Table I: by default (variant A) the reward is the raw confidence drop $C(y_{true}|\mathbf{x}) - C(y_{true}|\mathbf{x}')$, which is positive when the adversarial document reduces the target's confidence in the true class; variant B passes this drop through a sigmoid; and variant C weights the drop by the original confidence $C(y_{true}|\mathbf{x})$. Unless stated otherwise, r_{adv} denotes variant A.

To encourage semantic preservation, we additionally define

$$r_{sem}(\mathbf{x}', \mathbf{x}) = USE(\mathbf{x}', \mathbf{x}), \quad (3)$$

where $USE(\mathbf{x}', \mathbf{x})$ measures the semantic similarity between $\mathbf{x}$ and $\mathbf{x}'$ using the Universal Sentence Encoder [26], a model trained on large-scale textual data pairs for general semantic similarity. Higher values indicate greater similarity.

We combine both rewards via a weighted average:

$$r(\mathbf{x}', \mathbf{x}) = \alpha \cdot r_{adv}(\mathbf{x}', \mathbf{x}) + (1 - \alpha) \cdot r_{sem}(\mathbf{x}', \mathbf{x}), \quad (4)$$

where α is an adjustable parameter controlling the trade-off between adversarial effectiveness and semantic similarity.

B. Attacker Training

We define our reward objective R_θ as the expected value of $r(\mathbf{x}', \mathbf{x})$ over the distribution $\mathcal{D}$ of documents $\mathbf{x}$ and the conditional distribution $A_\theta(\cdot|\mathbf{x})$ of $\mathbf{x}'$:

$$R_\theta = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [\mathbb{E}_{\mathbf{x}' \sim A_\theta(\cdot|\mathbf{x})} [r(\mathbf{x}', \mathbf{x})]]. \quad (5)$$

For detailed implementation, we apply the likelihood-ratio trick and estimate the gradient by Monte Carlo sampling, restricting token replacements to the target positions $S(\mathbf{x}_i)$ to optimize R_θ :

$$\begin{aligned} R_\theta &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [\mathbb{E}_{\mathbf{x}' \sim A_\theta(\cdot|\mathbf{x})} [r(\mathbf{x}', \mathbf{x})]] \\ &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\sum_{\mathbf{x}'} A_\theta(\mathbf{x}' | \mathbf{x}) r(\mathbf{x}', \mathbf{x}) \right]. \end{aligned} \quad (6)$$

Since R_θ is a differentiable function of θ , we obtain its gradient with the likelihood-ratio (log-derivative) trick:

$$\begin{aligned} \nabla R_\theta &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\sum_{\mathbf{x}'} \nabla A_\theta(\mathbf{x}' | \mathbf{x}) r(\mathbf{x}', \mathbf{x}) \right] \\ &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\sum_{\mathbf{x}'} A_\theta(\mathbf{x}' | \mathbf{x}) \nabla \log A_\theta(\mathbf{x}' | \mathbf{x}) r(\mathbf{x}', \mathbf{x}) \right] \\ &\approx \frac{1}{b} \sum_{i=1}^b \nabla \log A_\theta(\mathbf{x}_i^* | \mathbf{x}_i) r(\mathbf{x}_i^*, \mathbf{x}_i). \end{aligned} \quad (7)$$

The final line applies Monte Carlo estimation over a batch of b source documents $\{\mathbf{x}_i\}_{i=1}^b$, drawing a single adversarial document $\mathbf{x}_i^* \sim A_\theta(\cdot | \mathbf{x}_i)$ per input. Because token replacements are permitted only at the target positions $j \in S(\mathbf{x}_i)$, this can be written as

$$\nabla R_\theta \approx \frac{1}{b} \sum_{i=1}^b \sum_{j \in S(\mathbf{x}_i)} \nabla \log A_\theta(t_{ij}^* | \mathbf{x}_i) r(\mathbf{x}_i^*, \mathbf{x}_i), \quad (8)$$

where t_{ij}^* is the adversarial token at position j in document $\mathbf{x}_i$. Following the Policy Gradient method [18], we perform gradient ascent on R_θ , which is equivalent to maximizing the reward-weighted log-likelihood objective:

$$L_\theta = \frac{1}{b} \sum_{i=1}^b \sum_{j \in S(\mathbf{x}_i)} [r(\mathbf{x}_i^*, \mathbf{x}_i) \log A_\theta(t_{ij}^* | \mathbf{x}_i)], \quad (9)$$

where b is the training batch size (the number of source documents sampled per gradient update), $S(\mathbf{x}_i)$ is the set of positions perturbed in document $\mathbf{x}_i$, t_{ij}^* is the adversarial token sampled from A_θ at position j , and $\mathbf{x}_i^*$ is the resulting perturbed document.

C. Attacking

The overall attacking procedure follows a structure similar to the training phase, with two primary distinctions.

1) *Identifying Influential tokens*: Let $\mathbf{x} = [t_1, t_2, \dots, t_m]$ denote a source document, and let y_{true} denote its task-specific true label. In the first step, we quantify the influence of tokens t_n ($n = 1, \dots, m$) in $\mathbf{x}$ as

$$I(t_n) = C(y_{\text{true}}|\mathbf{x}) - C(y_{\text{true}}|\mathbf{x} \setminus t_n),$$

Thus, a token t_n with a higher $I(t_n)$ value tends to have a greater influence on model prediction outcomes when it is dropped (or masked). This step is illustrated in the bottom part of the **Attacking Mechanism** panel in Figure 2. Heuristically, to enhance the chance of a successful attack, we replace these high-influence tokens with adversarial candidates. We define the target index set $S(\mathbf{x})$ as follows, with K (the *edit budget* — number of tokens perturbed per document) as a hyperparameter:

$$S(\mathbf{x}) = \{n : t_n \in \mathbf{x}, \text{rank}(I(t_n)) \leq K\}.$$

2) *Iterative Attacking*: This step, which generates the adversarial document $\mathbf{x}'$, is depicted in the **Attacking Mechanism** panel in Figure 2. The trained attacker A_θ takes a source document $\mathbf{x}$ and generates, for each target token in $S(\mathbf{x})$, the top- N adversarial token candidates from the distribution $T_\theta(\cdot|\mathbf{x})$, where N is the per-position candidate budget. Given a set of adversarial candidates, the attacker iteratively replaces the target tokens in $S(\mathbf{x})$ with each candidate to form a new potential adversarial document. This procedure is greedy in nature, and we retain the candidates that tend to decrease the true class probability the most. The procedure continues iterating over the target tokens until the prediction has changed or the maximum allowed number of queries is reached and the attack fails.

IV. EXPERIMENT

We conduct experiments on both natural language inference and generative Q&A tasks to demonstrate the generalization of our attacker. Following prior work [8], [27], [28], we adopt the following metrics for fair comparison:

Post-attack Accuracy. The accuracy of the target model evaluated on the adversarial inputs produced by the attack: for the classification tasks, the fraction of perturbed inputs still assigned their correct label; for the generative setting, the fraction of responses still judged safe by the guard model. A lower value therefore indicates a more effective attack. This quantity underlies the adversarial reward r_{adv} that the attacker is trained to maximize (Section III-A).

Number of Queries. The average number of queries issued to the black-box target model to craft a single adversarial example. Fewer queries imply greater practicality under real-world cost or rate constraints, and also make the attack harder to detect.

USE. The cosine similarity between the original and perturbed texts, each embedded with the Universal Sentence Encoder [26]. It measures how well the perturbation preserves the original meaning, so a higher value denotes better semantic preservation. It is also used as the semantic-similarity reward r_{sem} during training (Section III-A).

Percentage of Perturbation. The proportion of tokens altered relative to the length of the original document. A lower percentage reflects a smaller, less conspicuous edit and hence less semantic distortion.

A. Implementation Details

The same RL-ATTACK implementation drives both natural language inference (NLI) and generative Q&A: the attacker architecture, the policy-gradient training objective (Eq. 9), and the identify-influential-tokens-then-substitute attack mechanism (Section III-C) are identical across the two tasks—the framework is universal. The two tasks differ in two respects: the adversarial reward and the criterion for a successful attack.

For NLI the target is a classifier: the reward is the drop in its true-label confidence $C(y_{\text{true}}|\cdot)$, and an attack succeeds when the prediction flips to any incorrect label.

For generative Q&A, the ultimate attack target is an LLM whose responses are assessed by a safety judge. To reduce training costs, we do not query the target LLM during attacker training. Instead, we train the attacker against Llama-Guard-4-12B, a lower-cost safety classifier designed to identify unsafe prompts. We hypothesize that optimizing the attacker against this specialized judge enables it to learn adversarial perturbations that transfer to jailbreak attacks on target LLMs, and we evaluate this transfer at attack time. Specifically, during training, Llama-Guard-4-12B provides the confidence $C(y_{\text{true}}|\cdot)$, defined as the probability assigned to a prompt’s ground-truth safety label, which is used to compute the adversarial reward. At attack time, the resulting adversarial prompt is submitted to the target LLM, and the generated response is evaluated by either Llama-Guard-4-12B or the OpenAI Moderation API. A jailbreak is considered successful when the active judge classifies the target LLM’s response as unsafe. This design still follows our generalized framework.

The attacker is a masked-language-model perturbation network built on BERT with its built-in masked-LM head. Of its 109.5M total parameters, only ≈ 0.6 M are trainable: the pretrained encoder and the tied token embeddings are frozen, and we learn only the masked-LM projection head (a $768 \rightarrow 768$ transform with an output bias over the 30,522-token vocabulary). Given a document with K masked positions, the head emits a distribution over the vocabulary at each position, from which the attack draws the top- N candidate substitutions per position at inference. The maximum sequence length is 512 tokens.

B. Evaluation on Natural Language Inference

Datasets and splits. We conduct untargeted adversarial attacks aiming to induce misclassification to any incorrect label on the standard SNLI [31] and MNLI [32] benchmarks, with MNLI consisting of matched and mismatched tracks. These datasets have pre-split train, validation, and evaluation subsets.

For each NLI benchmark, the train split is used to train the attacker against the corresponding BERT classifier, the validation split is used to choose a checkpoint during training, and the evaluation set is used to report performance.

TABLE II
ADVERSARIAL ATTACK RESULTS. PARENTHETICAL VALUES ARE STANDARD ERRORS COMPUTED FROM 10 VALIDATION FOLDS.

Dataset	Target Model	Method	Post-attack accuracy (%)	# of queries	USE	% of perturbation
SNLI	BERT	RL-ATTACK	2.03 (± 0.54)	13	0.65	5.1
		BERT (untrained)	5.73	18	0.60	6.0
		TextFooler [11]	4.00	60	0.45	18.5
		BERT-ATTACK [8]	5.73	16	0.40	12.4
		SSPAttack (hard-label) [27]	1.30	1000	0.54	15.9
		HQA-Attack (hard-label) [28]	1.30	1000	0.54	15.9
		TextHoaxer (hard-label) [29]	1.30	1000	0.39	16.6
		LeapAttack (hard-label) [30]	1.30	1000	0.35	19.9
MNLI matched	BERT	RL-ATTACK	0.90 (± 0.30)	15	0.70	3.2
		BERT (untrained)	3.00	14	0.58	4.5
		TextFooler	9.60	78	0.57	15.2
		BERT-ATTACK	7.90	19	0.55	8.8
		SSPAttack (hard-label)	2.90	1000	0.64	12.0
		HQA-Attack (hard-label)	2.90	1000	0.64	12.1
		TextHoaxer (hard-label)	2.90	1000	0.53	12.7
		LeapAttack (hard-label)	2.90	1000	0.49	15.7
MNLI mismatched	BERT	RL-ATTACK	0.80 (± 0.28)	16	0.73	3.0
		BERT (untrained)	2.10	32	0.63	4.2
		TextFooler	8.3	86	0.58	14.6
		BERT-ATTACK	7.00	24	0.53	4.6
		SSPAttack (hard-label)	1.70	1000	0.65	11.5
		HQA-Attack (hard-label)	1.70	1000	0.65	11.5
		TextHoaxer (hard-label)	1.70	1000	0.54	12.5
		LeapAttack (hard-label)	1.70	1000	0.50	15.1

Training setup. During training, we allow the attack to modify tokens in both the premise and the hypothesis. This encourages the model to consider the full context of the document and learn more comprehensive adversarial strategies. However, during evaluation, we follow prior work [8], [27], [28] and attack the hypothesis only for the purpose of fair comparison.

Training and attack parameters. Across many training hyperparameter settings, our attacks efficiently reduce post-attack accuracy. We do not observe notably stronger performance when tuning the training hyperparameters. We therefore choose the final configuration of $\alpha = 1$ and $K = 3$ across all NLI tasks. This small edit budget helps achieve better semantic preservation while remaining sufficient to achieve low post-attack accuracy. For the attack hyperparameter, we set $N = 40$ for all NLI tasks. We observe that this relatively small per-position candidate budget already reduces post-attack accuracy to approximately 2% or lower as shown in Table II. We therefore choose this small value to use fewer queries and better preserve semantics.

We will discuss the hyperparameters for the generative Q&A task in greater detail in the following section, since we observed a more obvious performance changes when tuning parameters for it.

Main results. As detailed in Table II, RL-ATTACK consistently achieves the lowest post-attack accuracies among the soft-label attacks, with all three values around 2% or below, outperforming the soft-label baselines. Relative to the BERT (untrained) baseline, RL-ATTACK lowers post-attack accuracy on all three NLI evaluation tracks. We hypothesize that these gains result from training the attacker from a vanilla BERT

initialization.

We also include hard-label attacks to position our results against the broader adversarial-attack literature, but they should not be read as a fair like-for-like comparison: hard-label methods face a different feedback setting because they do not access predicted-label probabilities, and the reported hard-label baselines use much larger query budgets. On SNLI, several hard-label methods attain lower raw post-attack accuracy than RL-ATTACK, but only with 1000 queries and lower semantic similarity; on MNLI matched and mismatched, RL-ATTACK has the lowest raw post-attack accuracy while also using far fewer queries. While the semantic similarity—measured by the USE score—varies across datasets, RL-ATTACK keeps the query count and perturbation rate low given the strength of the attack. Across all datasets, it requires, on average, only 13 to 16 queries to successfully attack a hypothesis, with perturbation rates largely remaining around 5% or below.

C. Evaluation on Generative Q&A

Datasets and splits. We use the in-the-wild jailbreak-prompt corpus [33] and AdvBench [16]. We partition the in-the-wild dataset into three disjoint splits: *train*, *traindev*, *evaldev*. The *train* split is used to train the attacker, *traindev* is used for checkpoint selection, *evaldev* is used to select the attacking hyperparameters. The same trained checkpoint is evaluated on AdvBench since AdvBench itself is a small dataset that is built specifically for robustness evaluation purpose. Table IV summarizes the splits.

TABLE III

ADVERSARIAL ATTACK RESULTS ON **ADVBENCH**. POST-ATTACK ACCURACY IS THE PERCENTAGE OF RESPONSES JUDGED SAFE BY THAT RUN’S JUDGE. PARENTHETICAL VALUES ARE STANDARD ERRORS COMPUTED FROM 10 VALIDATION FOLDS.

Judge	Target Model	Method	Post-attack accuracy	# of queries	USE	% of perturbation
Llama Guard 4 12B	Llama 2 7B [25]	RL-ATTACK	82.0 (± 3.8)	27.7	0.98	2.0%
		PAIR [17]	98.0	56.0	0.60	N/A
		BERT (untrained)	87.0	28.3	0.98	1.0%
	GPT-3.5-Turbo	RL-ATTACK	11.5 (± 3.3)	16.8	0.98	4.5%
		PAIR	28.0	29.6	0.44	N/A
		BERT (untrained)	15.5	15.5	0.97	4.7%
	GPT-5.4-mini	RL-ATTACK	92.6 (± 2.6)	29.0	0.99	1.1%
		PAIR	77.6	52.6	0.48	N/A
		BERT (untrained)	97.7	29.5	0.98	2.1%
OpenAI Moderation API	Llama 2 7B [25]	RL-ATTACK	81.0 (± 3.7)	26.9	0.99	1.4%
		PAIR	88.0	29.3	0.53	N/A
		BERT (untrained)	85.3	26.9	0.97	1.6%
	GPT-3.5-Turbo	RL-ATTACK	42.0 (± 4.9)	33.9	0.96	4.1%
		PAIR	40.0	46.6	0.46	N/A
		BERT (untrained)	61.5	47.0	0.96	4.4%
	GPT-5.4-mini	RL-ATTACK	84.0 (± 3.7)	26.2	0.99	1.1%
		PAIR	97.0	56.8	0.46	N/A
		BERT (untrained)	84.4	28.0	0.98	0.7%

TABLE IV

GENERATIVE Q&A DATA SPLITS. THE ATTACKER IS TRAINED ONLY ON THE IN-THE-WILD TRAIN SPLIT, BUT IS VALUATED ON BOTH THE IN-THE-WILD AND ADVBENCH EVAL SETS.

Dataset	Split	Size	Purpose
in-the-wild	train	16,776	Train the attacker (1,034 unsafe + 15,742 safe)
in-the-wild	traindev	50	Checkpoint selection at best traindev reward
in-the-wild	evaldev	100	Choose attacking hyperparameter
AdvBench	all	500	Test

Target models. We evaluate the vulnerability of Llama 2 7B [25], GPT-3.5-Turbo, and GPT-5.4-mini. These models are selected to ensure a fair comparison with PAIR [17] and to assess transferability across model scales. We do not evaluate more advanced proprietary models (e.g., GPT-5, Claude, Gemini Pro): their providers deploy aggressive, automated account-banning mechanisms, so issuing even a handful of adversarial (policy-violating) prompts risks suspending or banning the account, which makes systematic and repeatable evaluation infeasible.

Reward and judge models. Section IV-A summarizes the reward and success criterion for generative targets. Here, we specify how the judge confidence instantiates the shared objective. During training, Llama-Guard-4-12B [34] provides the true-label confidence $C(y_{\text{true}} | \cdot)$. The three reward variants (Table I), the semantic term r_{sem} (USE cosine similarity), and the combined reward $r = \alpha r_{\text{adv}} + (1 - \alpha)r_{\text{sem}}$ are exactly as defined in Section III-A, with $\alpha \in \{0.0, 0.1, \dots, 1.0\}$ sweeping the trade-off between attack effectiveness and semantic similarity.

At attack time, we first evaluate RL-ATTACK using Llama-Guard-4-12B, the same judge used during training and the judge adopted by PAIR. We additionally use the OpenAI Moderation API as a supplementary judge to test whether the attacker trained with the Llama Guard reward generalizes when attack success is determined by a different safety model. The Moderation API is never used during training.

Training and attacking. We optimize only the projection head using Adam (learning rate 0.01, $\beta = (0.9, 0.98)$) with batch size $b = 16$ for one epoch ($\approx 1,049$ steps over the 16,776 training prompts). We check the held-out DEV reward (the same objective on the 50-prompt traindev split) every 100 steps and save the checkpoint whenever it improves. The training hyperparameters are chosen by grid search over reward $\in \{A, B, C\} \times \alpha \in \{0.0, \dots, 1.0\} \times K \in \{3, 5, 10\}$ (99 configurations, with the candidate budget fixed at $N = 1$ during the search), each scored by 10-fold cross-validation on traindev (Figure 3); the edit budget K ($= \text{num_doc_masks}$) is a training hyperparameter, whereas the candidate budget N ($= \text{samples_per_tok}$) is fixed only at attack time. At attack time, we run a greedy token-substitution search with edit budget K and search budget N ; a document is successfully attacked when the target LLM’s response (temperature 0, max_new_tokens= 96) is judged unsafe by the active judge, and we report accuracy after attack.

Hyperparameter sensitivity and training effect. Fig. 3 summarizes 99 configurations spanning three reward variants, $\alpha \in \{0.0, 0.1, \dots, 1.0\}$, and $K \in \{3, 5, 10\}$. Their traindev post-attack accuracies range from 0.64 to 0.80 (mean 0.71, SD 0.035), with no consistent advantage for any reward variant, value of α , or edit budget. We therefore treat the grid as a descriptive sensitivity analysis, not evidence of a unique optimum.

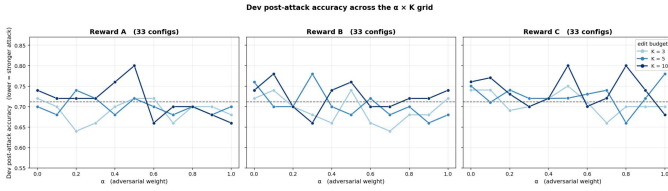


Fig. 3. The post-attack-accuracy on in-the-wild traindev set. Each point represent a α - K training hyperparameter combination. No consistent trend is observed.

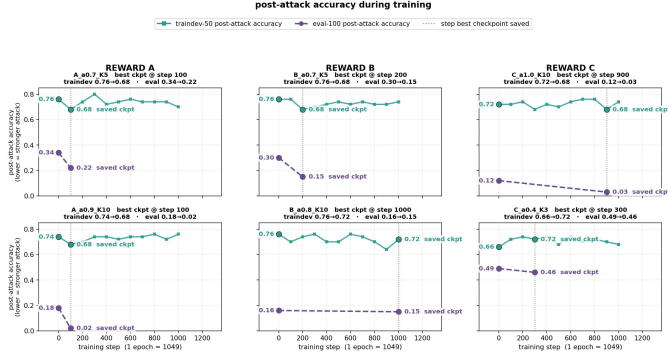


Fig. 4. Post-attack accuracy over training for six representative hyperparameter combinations under three types of rewards.

Fig. 4 compares the untrained attacker with the selected checkpoint for six representative configurations. Evaluation post-attack accuracy decreases in all six, from a marginal $0.16 \rightarrow 0.15$ to a substantial $0.18 \rightarrow 0.02$, while traindev accuracy decreases in five. In the remaining case, traindev accuracy increases from 0.66 to 0.72 even as evaluation accuracy decreases from 0.49 to 0.46. Moreover, four checkpoints with the same traindev accuracy of 0.68 yield evaluation accuracies from 0.02 to 0.22. Thus, training strengthens the attack in these examples, but traindev accuracy does not reliably rank held-out performance.

We therefore use reward variant A with $\alpha = 1$ and $K = 3$ as a simple, competitive attacker-training configuration with the default reward and smallest tested edit budget, without claiming a unique optimum.

Attack-time candidate budget (N). During attacker training and training-hyperparameter evaluation, we hold N at 1. At attack time, we observe diminishing returns as N increases. We set $N = 40$ in all generative LLM attack settings because it provides a good trade-off between post-attack accuracy and query cost.

Main results. Table III reports attacks on three target models under two independent judges. Only Llama Guard 4 12B is used for training the attacker, OpenAI Moderation API is used to demonstrate the generalization of our attacker. Compared with the optimization-based PAIR baseline, RL-ATTACK uses fewer queries and achieves substantially higher semantic similarity in all six settings: its USE scores range from 0.96 to 0.99, compared with 0.44 to 0.60 for PAIR. RL-ATTACK also achieves lower post-attack accuracy than

PAIR in four of the six settings. The two exceptions are GPT-5.4-mini under Llama Guard (92.6% for RL-ATTACK versus 77.6% for PAIR) and GPT-3.5-Turbo under the Moderation API (42.0% versus 40.0%).

Relative to the untrained BERT attacker, RL-ATTACK achieves a lower post-attack-accuracy point estimate in all six settings while maintaining comparable USE scores and queries efficiency. These results support the conclusion that reinforcement-learning training generally improves attack effectiveness without sacrificing semantic preservation or query efficiency, although the improvement over the untrained attacker is small for GPT-5.4-mini under the Moderation API (84.0% versus 84.4%). GPT-5.4-mini remains the most difficult target overall in our setting.

V. CONCLUSION

We presented RL-ATTACK, a trainable black-box adversarial framework that leverages reinforcement learning to learn target-model-specific transformations. Experiments on natural language inference and generative Q&A tasks show that RL-ATTACK produces strong attacks with high semantic preservation at low query cost in most evaluation settings, and improves over an untrained attacker. Our conclusions are limited by the scope of our evaluation settings, the modest sizes of the evaluation sets, and the lack of an explicit grammaticality constraint on adversarial documents.

ACKNOWLEDGMENT

This work was supported by the Financial Services Analytics (FSAN) Ph.D. Program at the University of Delaware.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.
- [2] M. V. Koroteev, “Bert: a review of applications in natural language processing and understanding,” *arXiv preprint arXiv:2103.11943*, 2021.
- [3] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A robustly optimized BERT pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019. [Online]. Available: <https://arxiv.org/abs/1907.11692>
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992. [Online]. Available: <https://aclanthology.org/D19-1410/>
- [5] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [6] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican *et al.*, “Gemini: a family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.
- [7] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [8] L. Li, R. Ma, Q. Guo, X. Xue, and X. Qiu, “Bert-attack: Adversarial attack against bert using bert,” 2020.
- [9] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (llm) security and privacy: The good, the bad, and the ugly,” *High-Confidence Computing*, p. 100211, 2024.

- [10] M. Alzantot, Y. Sharma, A. Elgohary, B.-J. Ho, M. Srivastava, and K.-W. Chang, "Generating natural language adversarial examples," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds. Brussels, Belgium: Association for Computational Linguistics, Oct.-Nov. 2018, pp. 2890–2896. [Online]. Available: <https://aclanthology.org/D18-1316/>
- [11] D. Jin, Z. Jin, J. T. Zhou, and P. Szolovits, "Is bert really robust? a strong baseline for natural language attack on text classification and entailment," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, 2020, pp. 8018–8025.
- [12] D. Li, Y. Zhang, H. Peng, L. Chen, C. Brockett, M.-T. Sun, and B. Dolan, "Contextualized perturbation for textual adversarial attack," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 5053–5069. [Online]. Available: <https://aclanthology.org/2021.naacl-main.400/>
- [13] W. Hu and Y. Tan, "Black-box attacks against rnn based malware detection algorithms," in *Workshops at the Thirty-Second AAAI Conference on Artificial Intelligence*, Jun. 2018.
- [14] J. Gao, J. Lanchantin, M. L. Soffa, and Y. Qi, "Black-box generation of adversarial text sequences to evade deep learning classifiers," in *2018 IEEE Security and Privacy Workshops (SPW)*. IEEE, May 2018, pp. 50–56.
- [15] J. Li, S. Ji, T. Du, B. Li, and T. Wang, "Textbugger: Generating adversarial text against real-world applications," *arXiv preprint arXiv:1812.05271*, 2018.
- [16] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023.
- [17] P. Chao, A. Robey, E. Dobriban, H. Hassani, G. J. Pappas, and E. Wong, "Jailbreaking black box large language models in twenty queries," in *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 2025, pp. 23–42.
- [18] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," *Advances in neural information processing systems*, vol. 12, 1999.
- [19] Y. Keneshloo, T. Shi, N. Ramakrishnan, and C. K. Reddy, "Deep reinforcement learning for sequence to sequence models," *arXiv preprint arXiv:1805.09461*, 2018.
- [20] R. Paulus, "A deep reinforced model for abstractive summarization," *arXiv preprint arXiv:1705.04304*, 2017.
- [21] Y. Wu and B. Hu, "Learning to extract coherent summary via deep reinforcement learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, 2018.
- [22] Y. Gao, C. M. Meyer, and I. Gurevych, "Preference-based interactive multi-document summarisation," *Information Retrieval Journal*, vol. 23, pp. 555–585, 2020.
- [23] K. Nguyen, H. Daumé III, and J. Boyd-Graber, "Reinforcement learning for bandit neural machine translation with simulated human feedback," in *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017. [Online]. Available: <https://aclanthology.org/D17-1153/>
- [24] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.
- [25] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [26] D. Cer, Y. Yang, S.-y. Kong, N. Hua, N. Limtiaco, R. St. John, N. Constant, M. Guajardo-Cespedes, S. Yuan, C. Tar, B. Strope, and R. Kurzweil, "Universal sentence encoder for English," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, E. Blanco and W. Lu, Eds. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 169–174. [Online]. Available: <https://aclanthology.org/D18-2029/>
- [27] H. Liu, Z. Xu, X. Zhang, X. Xu, F. Zhang, F. Ma, H. Chen, H. Yu, and X. Zhang, "Sspattack: a simple and sweet paradigm for black-box hard-label textual adversarial attack," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, 2023, pp. 13 228–13 235.
- [28] H. Liu, Z. Xu, X. Zhang, F. Zhang, F. Ma, H. Chen, H. Yu, and X. Zhang, "Hqa-attack: toward high quality black-box hard-label adversarial attack on text," *Advances in Neural Information Processing Systems*, vol. 36, pp. 51 347–51 358, 2023.
- [29] M. Ye, C. Miao, T. Wang, and F. Ma, "Textthoaxer: Budgeted hard-label adversarial attacks on text," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, 2022, pp. 3877–3884.
- [30] M. Ye, J. Chen, C. Miao, T. Wang, and F. Ma, "Leapattack: Hard-label adversarial attack on text via gradient-based optimization," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022, pp. 2307–2315.
- [31] S. Bowman, G. Angeli, C. Potts, and C. D. Manning, "A large annotated corpus for learning natural language inference," in *Proceedings of the 2015 conference on empirical methods in natural language processing*, 2015, pp. 632–642.
- [32] A. Williams, N. Nangia, and S. Bowman, "A broad-coverage challenge corpus for sentence understanding through inference," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, M. Walker, H. Ji, and A. Stent, Eds. New Orleans, Louisiana: Association for Computational Linguistics, Jun. 2018, pp. 1112–1122. [Online]. Available: <https://aclanthology.org/N18-1101/>
- [33] X. Shen, Z. Chen, M. Backes, Y. Shen, and Y. Zhang, "Do Anything Now": Characterizing and Evaluating In-The-Wild Jailbreak Prompts on Large Language Models," in *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2024.
- [34] H. Inan *et al.*, "Llama guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2023.